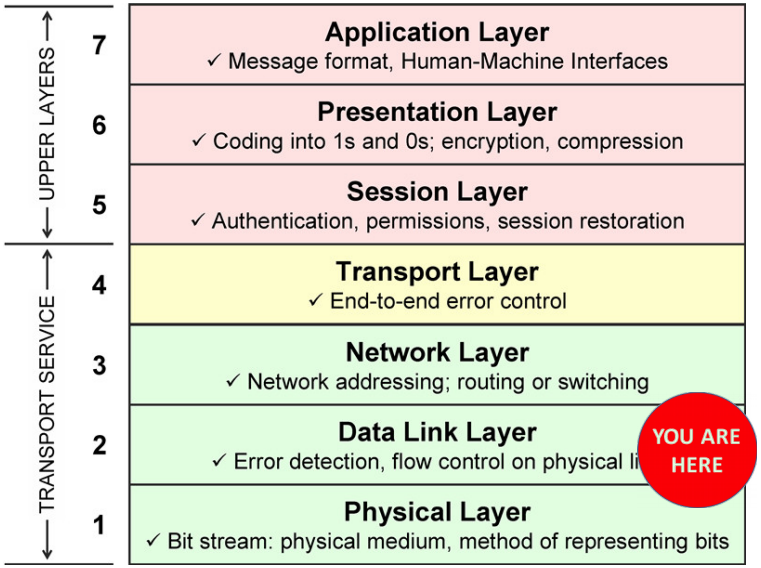
Nivelul legăturii de date

Cuprins

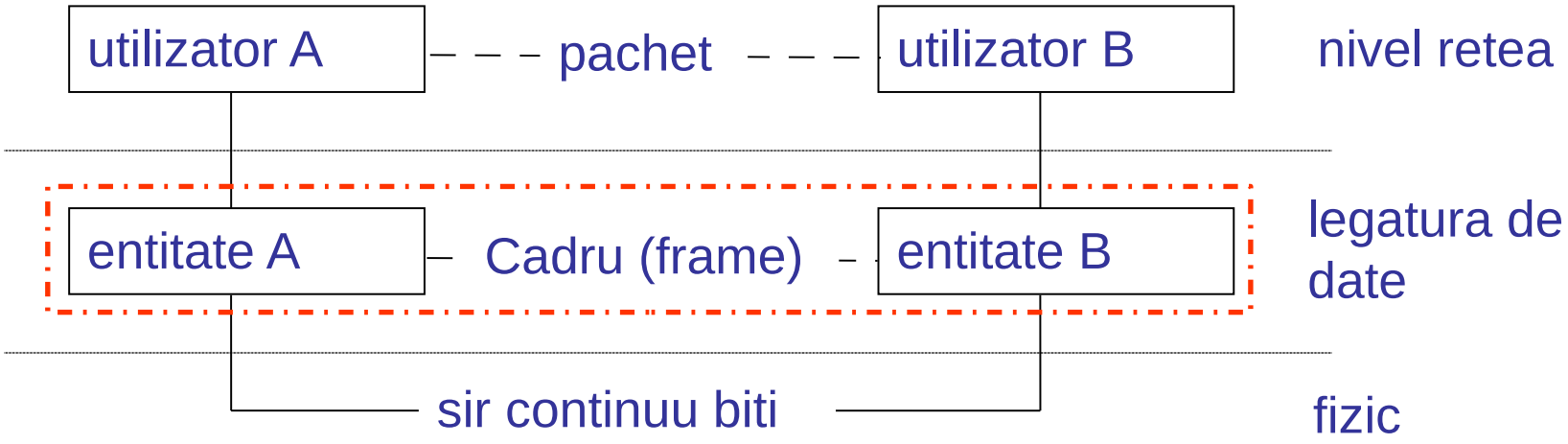
- Functiile legaturii de date
- Incadrarea datelor
- Transmisia transparenta
- Detectia erorilor de transmisie
- Protocoale start-stop
- Protocoale cu fereastra glisanta
- Exemple de protocoale: HDLC, PPP



Foarte importante!

- implemantarea protocoalelor (programarea)
- orientarea pe evenimente
- tratarea erorilor (nu exista in programarea “obisnuita”)
- mecanismul ferestrelor
- performanta (retransmitre neselectiva / selectiva)

Funcţiile nivelului legăturii de date



Încadrarea datelor

- nivelul fizic transmite un sir continuu de biti
- legatura de date partitioneaza sirul in cadre
- delimiteaza cadrele

Transmisia transparentă

- permite trimiterea in cadru a oricarei combinatii de biti
  - ex. care coincid cu delimitatorii cadrului



Controlul erorilor

- Coduri detectoare si corectoare de erori
- Mecanisme de protocol
  - mesaje de confirmare
  - ceasuri
  - numere de secvenţă

Controlul fluxului – protocol

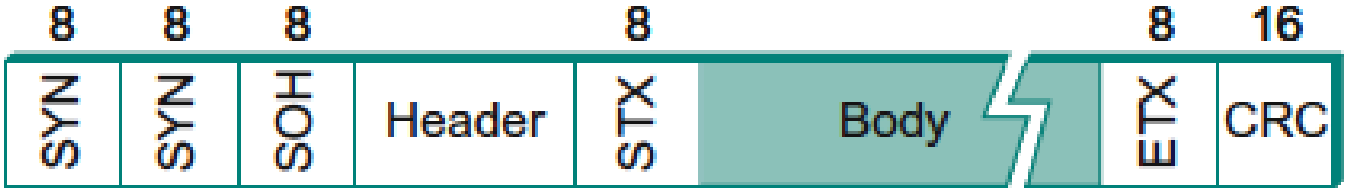
- mesaje de permisiune pentru transmiţător

Gestiunea legăturii – protocol

- stabilirea şi desfiinţarea legăturii
- re-iniţializare după erori
- configurarea legăturii (ex. legături multipunct)

Metode de încadrare a datelor

1) Caractere de control (BSC - Binary Synchronous Communication)



SOH - start of heading
STX - start of text
ETX - end of text
ETB - end of transmission block
EOT - end of transmission
ENQ - enquiry
ACK - acknowledge
NAK - not acknowledge
SYN - synchronous idle
DLE - data link escape
CRC - cyclic redundancy check

Metode de încadrare (2)

2) Numărarea caracterelor (DDCMP - Digital Data Communications Message Protocol)

SYN SYN SOH count flag resp seq address CRC data CRC

3) Indicatori de încadrare (HDLC - High Level Data Link Control)

flag address command data FCS flag

FCS – Frame Check Sequence

Transmisia transparentă

continut alfanumeric (nu contine coduri de control)
STX text ETX → OK!

Problema: continut binar:
STX text… ETX …text ETX → ETX fals ?!

Solutie:
- umplere cu caractere

Umplere cu caractere

- Dubleaza caracterele de control cu DLE

- Defineste combinatii admise
  - DLE STX – start text transparent
  - DLE ETX – sfarsit text transparent

    DLE STX text… ETX …text DLE ETX

- DLE in text – adauga inca un DLE la transmitere si elimina la receptie

  DLE STX ... DLE DLE ... DLE ETX

- Eroare la receptie DLE urmat de altceva decat STX, ETX, DLE

Umplere cu biţi

Datele de transmis includ un fals flag de terminare a cadrului

011011111101111111111010

01111110 011011111101111111111010 01111110

Solutia: adauga un zero dupa fiecare 5 unitati consecutive

01111110 011011111010111110111110010 01111110

si elimina la receptie zerourile adaugate

Adaugarea se face indiferent daca dupa 5 unitati urmeaza 0 sau 1

Simplifica regula receptorului: elimina zerouri aflate dupa 5 unitati

011011111010111110111110010

011011111101111111111010

Detecţia erorilor si corectarea erorilor

Descrierea problemei:

- Fie A = {0,1} un alfabet binar
- W_n multimea sirurilor w de lungime n peste A

  W= w[0] w[1]……...w[n-1]
- **Ponderea Hamming** a lui w = numarul de unitati (1) continute de w
- **Distanta Hamming**, d(u,v) dintre cuvintele u si v reprezinta numarul de simboluri diferite dintre cele doua. Este echivalent cu numarul minim de erori de 1 bit de care ar fi nevoie pentru a transforma unul din cuvinte in celalalt.

Detecţia erorilor si corectarea erorilor

Se considera ca pentru transmiterea de mesaje corecte se foloseste doar un subset din W_n

Multimea cuvintelor posibile W_n se imparte in:

- S_n – multimea cuvintelor cu sens
- F_n – multimea cuvintelor fara sens
- Pentru a detecta a cel mult r erori se alege S_n astfel ca:
  - d(u,v) >= r+1 pentru orice u,v din S_n
- Pentru a corecta cel mult r erori se alege S_n astfel ca:
  - d(u,v) >= 2*r+1 pentru orice u,v din S_n

Exemplu

S_{10} ={0000000000, 0000011111, 1111100000, 1111111111}

d(u,v) = 5 ====>>> putem detecta erori de maxim 4 biti
putem corecta erori de maxim 2 biti

Metoda Hamming

Se considera un sir de n biti numerotati de la 1 la n.
Bitii 1,2,4,8…..(puteri ale lui 2) sunt **biti de control**

Exemplu: pentru un payload de 8 biti este nevoie sa trimitem 4 biti de control (1,2,4,8)

Paritatea pentru bitii de control poate fi **para** sau **impara**, dupa urmatoarea regula:

Bit 1 – controleaza bitii 1,3,5,7,9,11
Bit 2 – controleaza bitii 2,3,6,7,10,11
Bit 4 – controleaza bitii 4,5,6,7,12
Bit 8 – controleaza bitii 8,9,10,11,12

Regula: fiecare bit k este controlat de bitii ale caror pozitii insumate dau k.

Corectarea erorilor

Codul Hamming se poate folosi pentru corectarea erorilor de 1 bit.

Se pot corecta erorile in rafala (en. burst) in situatia urmatoare:

- Matricea de biti este transmisa coloana cu coloana
- Se pot corecta erori in rafala dintr-o coloana daca esista un bit eronat pe fiecare linie.

Char.	ASCII	Check bits
H	1001000	00110010000
a	1100001	10111001001
m	1101101	11101010101
m	1101101	11101010101
i	1101001	01101011001
n	1101110	01101010110
g	1100111	01111001111
	0100000	10011000000
c	1100011	11111000011
o	1101111	10101011111
d	1100100	11111001100
e	1100101	00111000101

Order of bit transmission

Detecţia erorilor cu coduri polinomiale

k biţi de informaţie (date) ~ coeficientii unui polinom **i(X)** cu k termeni

Ex. k=5 10110 $i(X) = 1*X^4+0*X^3+1*X^2+1*X^1+0*X^0$

n-k biţi de control ~ polinom **r(X)** cu n-k termeni
concatenarea de **n** biţi ~ polinom de n termeni

$$\mathbf{w(X) = X^{(n-k)}i(X) + r(X)}$$

Pentru detectie erori
se foloseste un polinom generator **g(X)**
se calculeaza **r(X)** astfel ca **w(X)** sa fie multiplu de **g(X)**
se verifica daca proprietatea este pastrata la receptia cadrului
Calcul r(X):
$w(X) = g(X).q(X)$
$X^{(n-k)}i(X) + r(X) = g(X).q(X)$
$X^{(n-k)}i(X) = g(X).q(X) + r(X)$
r(X) este restul împărţirii lui $\mathbf{X^{(n-k)}}$ **i(X)** la **g(X)**

Calculul sumei de control

Frame : 1 1 0 1 0 1 1 0 1 1

Generator: 1 0 0 1 1

Message after 4 zero bits are appended: 1 1 0 1 0 1 1 0 1 1 0 0 0 0

Calculul sumei de control pentru un cod polinomial

10 biti informatie + 4 biti control

Imparte **11010110110000**

la 10011

$X^{(n-k)}\, i(X)$

$g(X)$

```
                1 1 0 0 0 0 1 0 1 0
          ------------------------------
10011   | 1 1 0 1 0 1 1 0 1 1 0 0 0 0
          1 0 0 1 1
          -----------
            1 0 0 1 1
            1 0 0 1 1
            ---------
              0 0 0 0 1
              0 0 0 0 0
              ---------
                0 0 0 1 0
                0 0 0 0 0
                ---------
                  0 0 1 0 1
                  0 0 0 0 0
                  ---------
                    0 1 0 1 1
                    0 0 0 0 0
                    ---------
                      1 0 1 1 0
                      1 0 0 1 1
                      ---------
                        0 1 0 1 0
                        0 0 0 0 0
                        ---------
                          1 0 1 0 0
                          1 0 0 1 1
                          ---------
                            0 1 1 1 0
                            0 0 0 0 0
                            ---------
                              1 1 1 0   <- Remainder
```

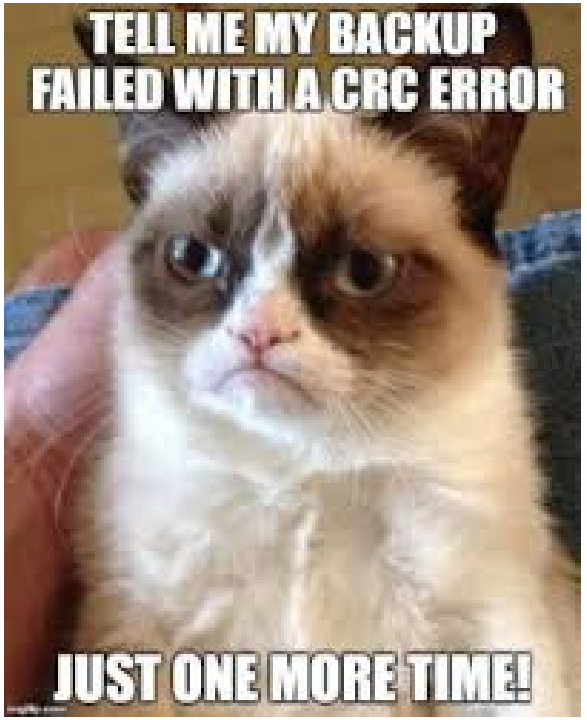
r(X) = rest împărţire X(n-k) i(X) la g(X)

Transmitted frame: 1 1 0 1 0 1 1 0 1 1 1 1 1 0

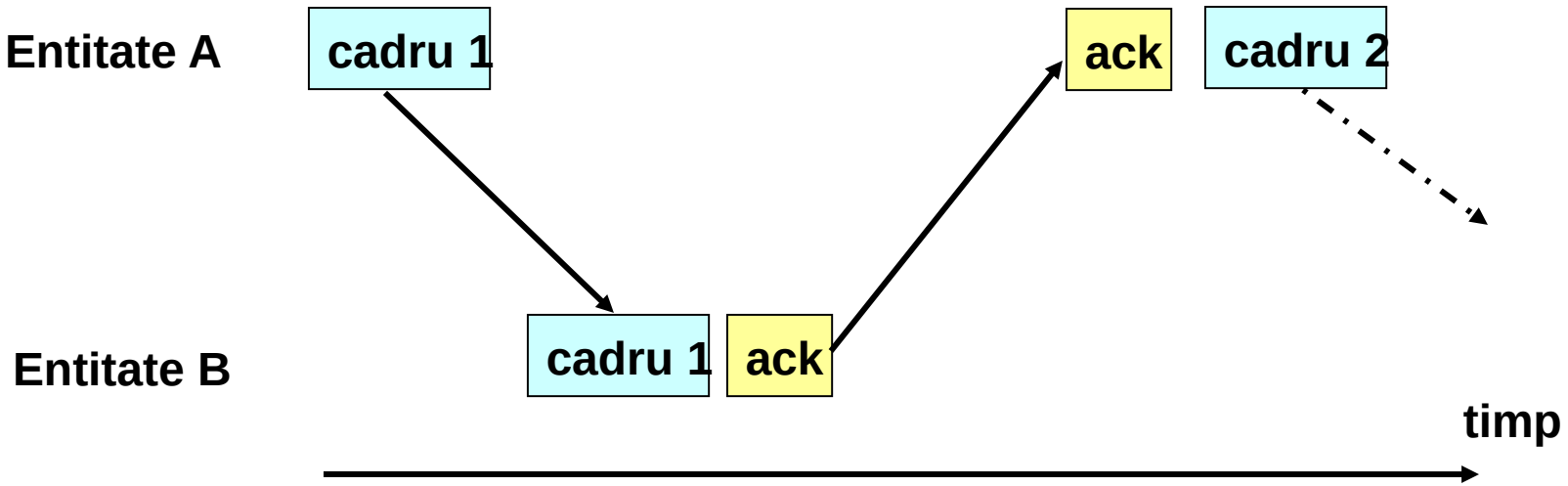
Ce erori pot fi detectate?

- Probabilitatea de detectie depinde de lungimea Codului cu Redundanta Ciclica
- CRC si sume de control pe
  - 8 biti detecteaza 99.6094% din erori (x^8+x^2+x+1 CRC-8-CCITT)
  - 16 biti detecteaza 99.9985% din erori ($x^{16}+x^{10}+x^8+x^7+x^3+1$ CRC-16)
  - 32 biti detecteaza 99.9999% din erori

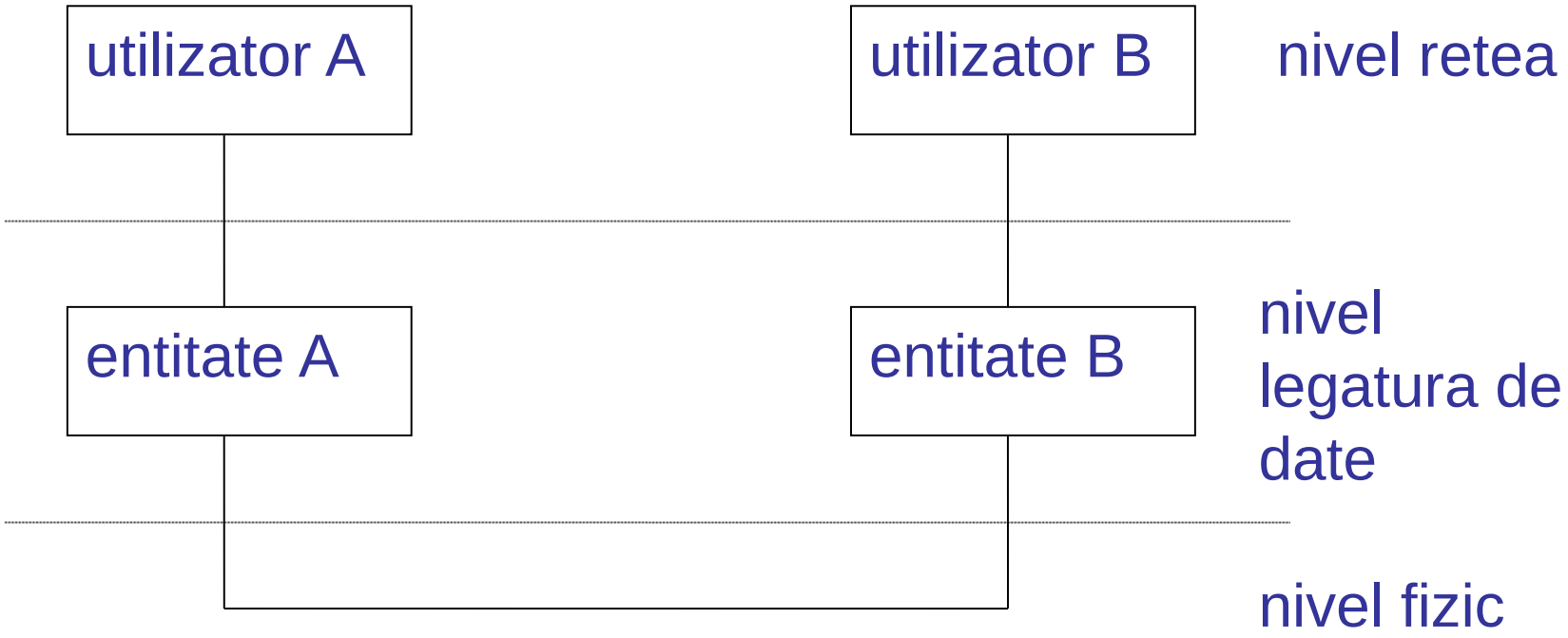
- In plus, CRC detecteaza 100% erori de
  - 1 bit;
  - 2 biti;
  - un numar impar de biti;
  - erori in rafala de lungimea codului CRC.



Protocoale start-stop pentru legătura de date



Configuraţia entităţilor de protocol



Formatul cadrelor

```
typedef unsigned int NrSecv;
typedef unsigned int word;

typedef enum FelCadru {data, ack, nak};
  //distinge intre cadre de date sau control
typedef struct {void *adresa;
                word lungime;
} pachet;
typedef struct {      //antet + date
    FelCadru fel;
    NrSecv secv, conf;
    pachet info;
} cadru;
```

Primitivele de serviciu

Serviciu furnizat nivelului superior (retea)

- preluarea unui pachet de la retea pentru transmitere pe canal

```
pachet DeLaRetea();
```

- livrarea către retea a unui pachet

```
void LaRetea (pachet);
```

Serviciu folosit de la nivelul inferior (fizic)

- trecerea unui cadru nivelului fizic pentru transmisie

```
void LaFizic (cadru);
```

- preluarea unui cadru de la nivelul fizic

```
cadru DeLaFizic();
```

Tratarea evenimentelor

Unele operatii ale entitatilor de protocol sunt declansate de evenimente de un anumit tip

```
typedef enum TipEven { SosireCadru,
        EroareControl,
        TimeOut,
        ReteaPregatita};
```

Entitatea intra in asteptarea producerii unui eveniment de tipul definit de enum, prin apelul unei operatii wait

```
TipEven wait();
```

La producerea unui eveniment de TipEven, entitatea primeste o informatie despre tipul evenimentului produs si este re-pusa in executie

Obs. functionalitatile ptr evenimente sunt asigurate de SO

Protocoale start-stop

Protocol simplex fara restrictii

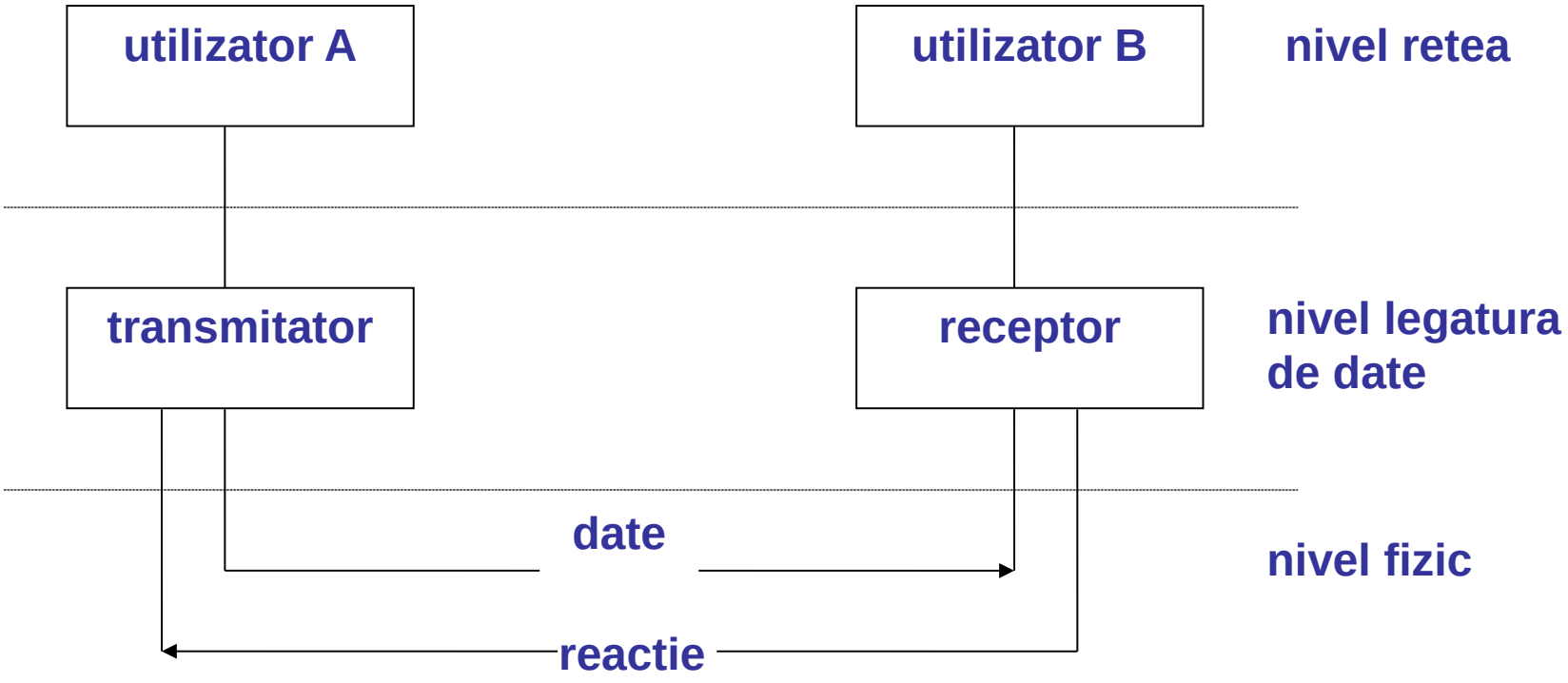
- utilizatorul A vrea să transmită date lui B folosind o legătură sigură, simplex;
- A reprezintă o sursă inepuizabilă de date;
- B reprezintă un consumator ideal;
- canalul fizic de comunicaţie este fără erori.

```
# define forever while(1)
// entitatea din sistemul transmitatorului
void transmit1(){
    cadru s;
    do{
        s.info = DeLaRetea();  //preia pachet
        LaFizic(s);            //transmite cadru
        } forever;
    }

    // entitatea din sistemul receptorului
    void recept1(){
        cadru r;
        TipEven even;
        do{
          even = wait();          //asteapta cadru
          r = DeLaFizic();        //primeste cadru
          LaRetea(r.info);        //preda pachet
        } forever;
    }
```

Protocol simplex start-stop

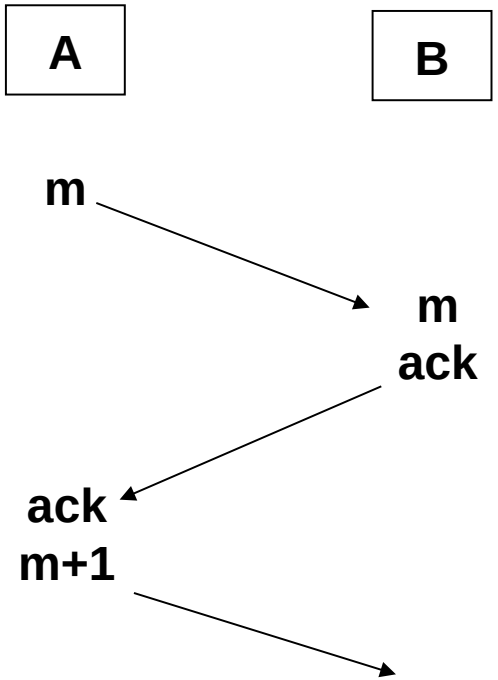
canal fara erori
dar utilizatorul B nu poate accepta date în orice ritm



```
void transmit2(){
   cadru s;
   TipEven even;
   do{
       s.info=DeLaRetea();
       LaFizic(s);
       even=wait();          //asteapta permisiunea
       } forever;
   }

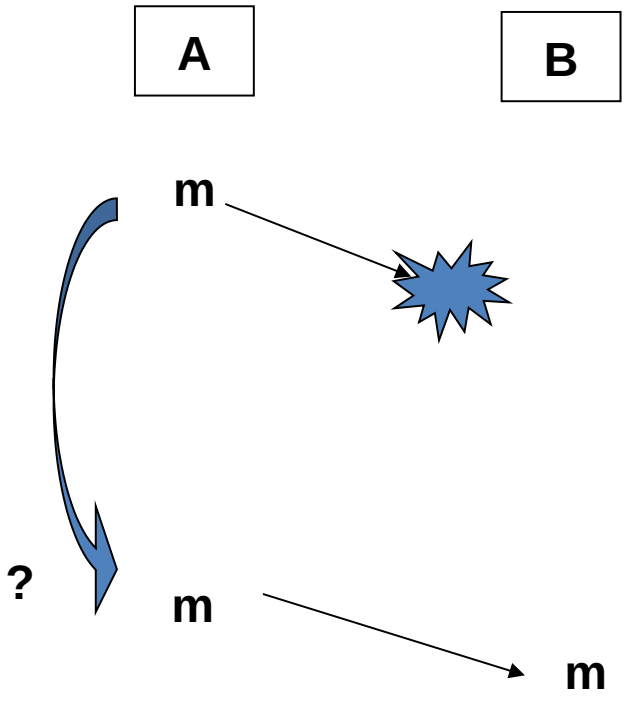
   void recept2(){
      cadru s,r;
      TipEven even;
      do{
       even=wait();          //poate fi doar SosireCadru
       r=DeLaFizic();
       LaRetea(r.info);
       LaFizic(s);           //transmite permisiunea
      } forever;
   }
```

Protocol simplex pentru un canal cu erori

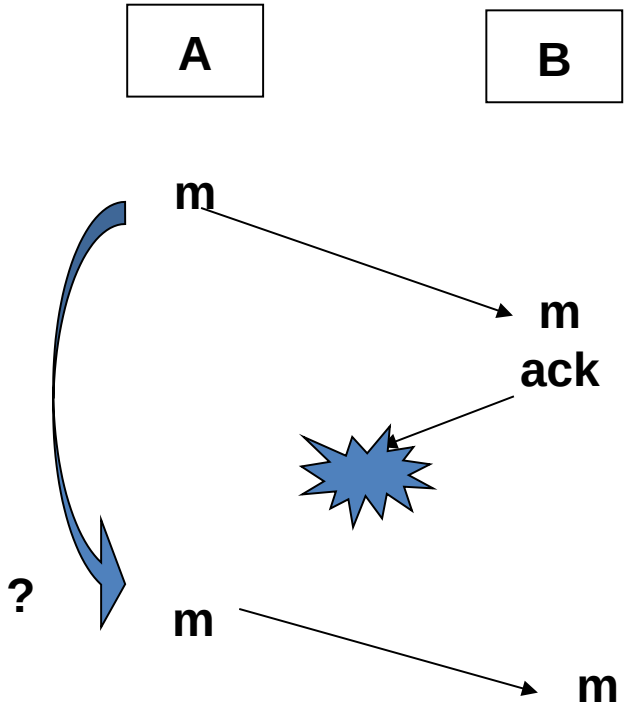


Entitaţi legătură de date

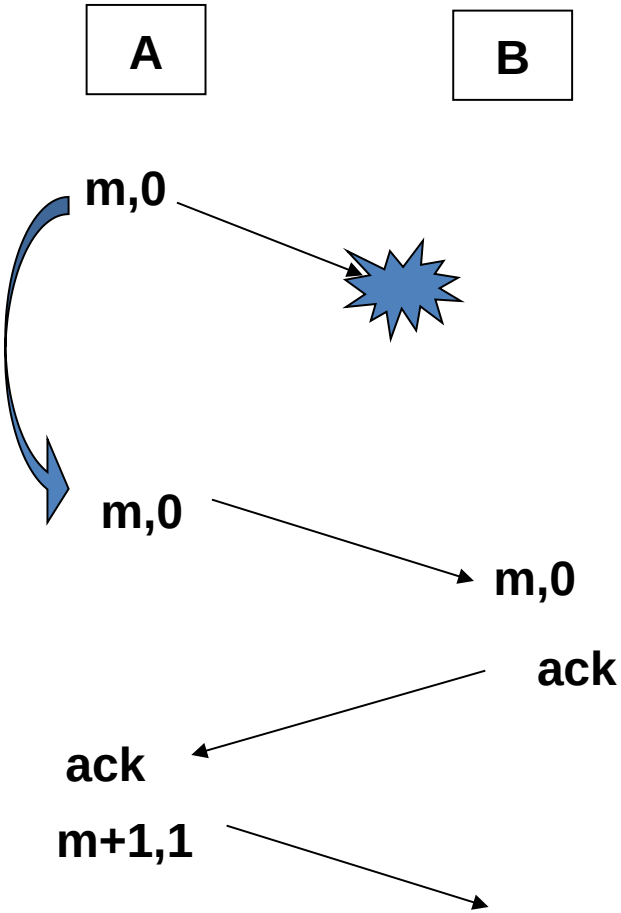
Transmisie corecta

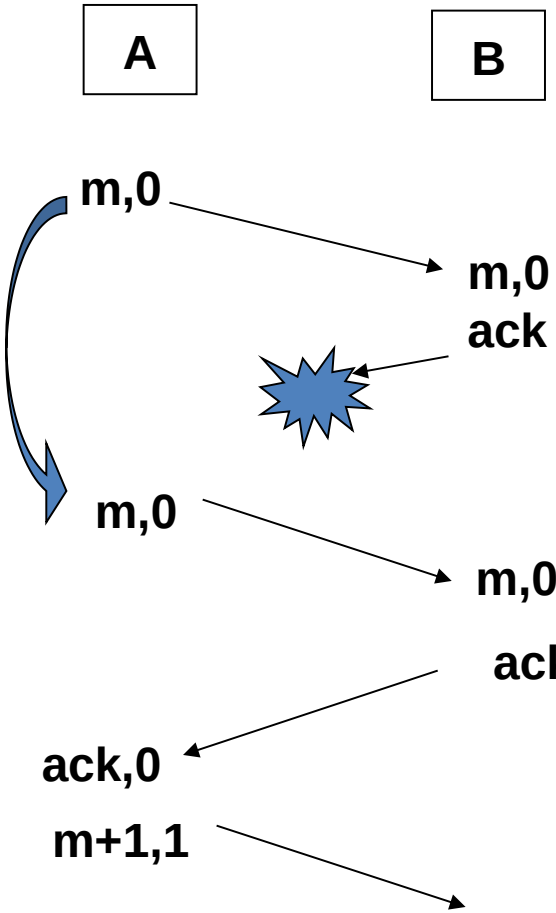


Pierdere m
La time-out A retransmite m
Care este acceptat corect de B



Pierdere ack
La time-out se retrimite m
Care este acceptat incorect, ca mesaj nou de B !





se adauga un numar de secventa
la time-out se re-transmite ultimul cadru
B accepta daca este corect

B ignora daca este dublura
dar re-trimite ack !

Protocol simplex pentru un canal cu erori (2)

Este nevoie de un ceas, asociat cu numarul cadrului trimis (NrSecv)

```
void StartCeas(NrSecv);
void StopCeas (NrSecv);
```

de evenimentul TimeOut

si de numere de secventa - cadrele succesive m, m+1, m+2 au numerele de secvenţa alternante 0, 1, 0 ... (protocol cu bit alternat)

fiecare cadru are campurile info si secv;
incrementarea se face prin functia generala de numarare ciclica:

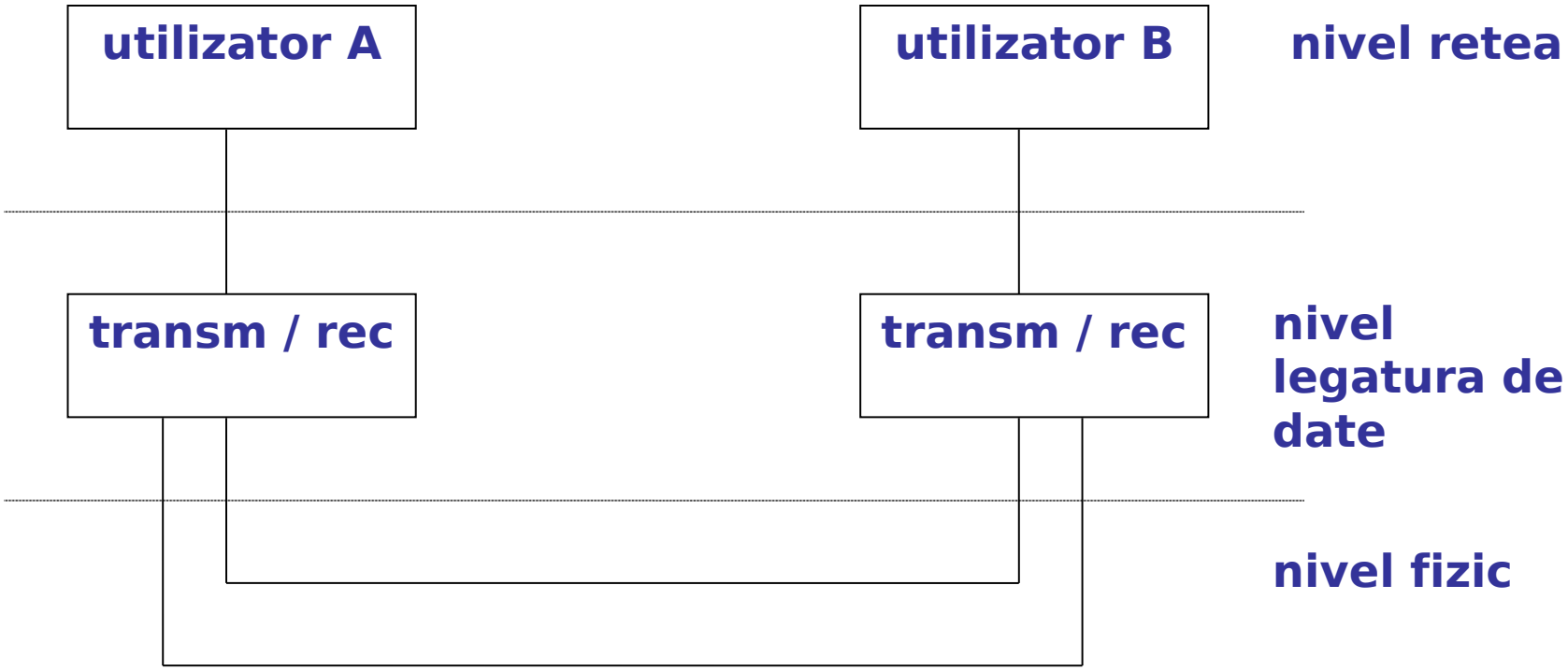
```
                    void inc (NrSecv&);
#define MaxSecv 1
void inc(NrSecv& k){
   k==MaxSecv ? k=0 : k++;
}
```

```
void transmit3() {
    NrSecv CadruUrmator=0;
    cadru s;
    TipEven even;
    s.info=DeLaRetea();     //pregateste un cadru
    do{
        s.secv=CadruUrmator;   //adauga nr secventa
        LaFizic(s);
        StartCeas(s.secv);
        even=wait();          // poate fi SosireCadru,
                            // TimeOut sau
                            // EroareControl
        if(even==SosireCadru) {   //confirmare intacta
            StopCeas(s.secv);
            s.info=DeLaRetea();
            inc(CadruUrmator);
        }
      }forever;
    }
```

```
void recept3(){
   NrSecv CadruAsteptat=0;
   cadru r,s;
   TipEven even;
   do{
       even=wait();  //SosireCadru sau EroareControl
       if(even==SosireCadru){
           r=DeLaFizic();
           if(r.secv==CadruAsteptat){
               LaRetea(r.info);     //cadru în secventa
               inc(CadruAsteptat);
           }
           LaFizic(s);   //transmite oricum confirmarea
       }
      }forever;
   }
```

Protocoale cu fereastră glisantă

Configuraţia generala



2 legaturi pentru date+confirmare

Preambul: Protocol cu numar de secventa de un bit

Fiecare entitate

are o secventa de initializare in care trimite un prim cadru

si realizează ciclic următoarele operaţii:

- receptia unui cadru,
- prelucrarea sirului de cadre receptionate,
- prelucrarea sirului de cadre transmise,
- transmiterea sau retransmiterea unui cadru impreuna cu confirmarea cadrului receptionat corect.

Fiecare cadru are campurile: info, secv, conf

```
void protocol4(){
    NrSecv CadruUrmator=0;   // 0 sau 1
    NrSecv CadruAsteptat=0;  // 0 sau 1
    cadru r,s;
    TipEven even;          //SosireCadru, TimeOut
                           //sau EroareControl

//pregateste cadru initial
    s.info=DeLaRetea();
    s.secv=CadruUrmator;
    s.conf=1-CadruAsteptat;
    LaFizic(s);          //transmite cadrul
    StartCeas(s.secv);
```

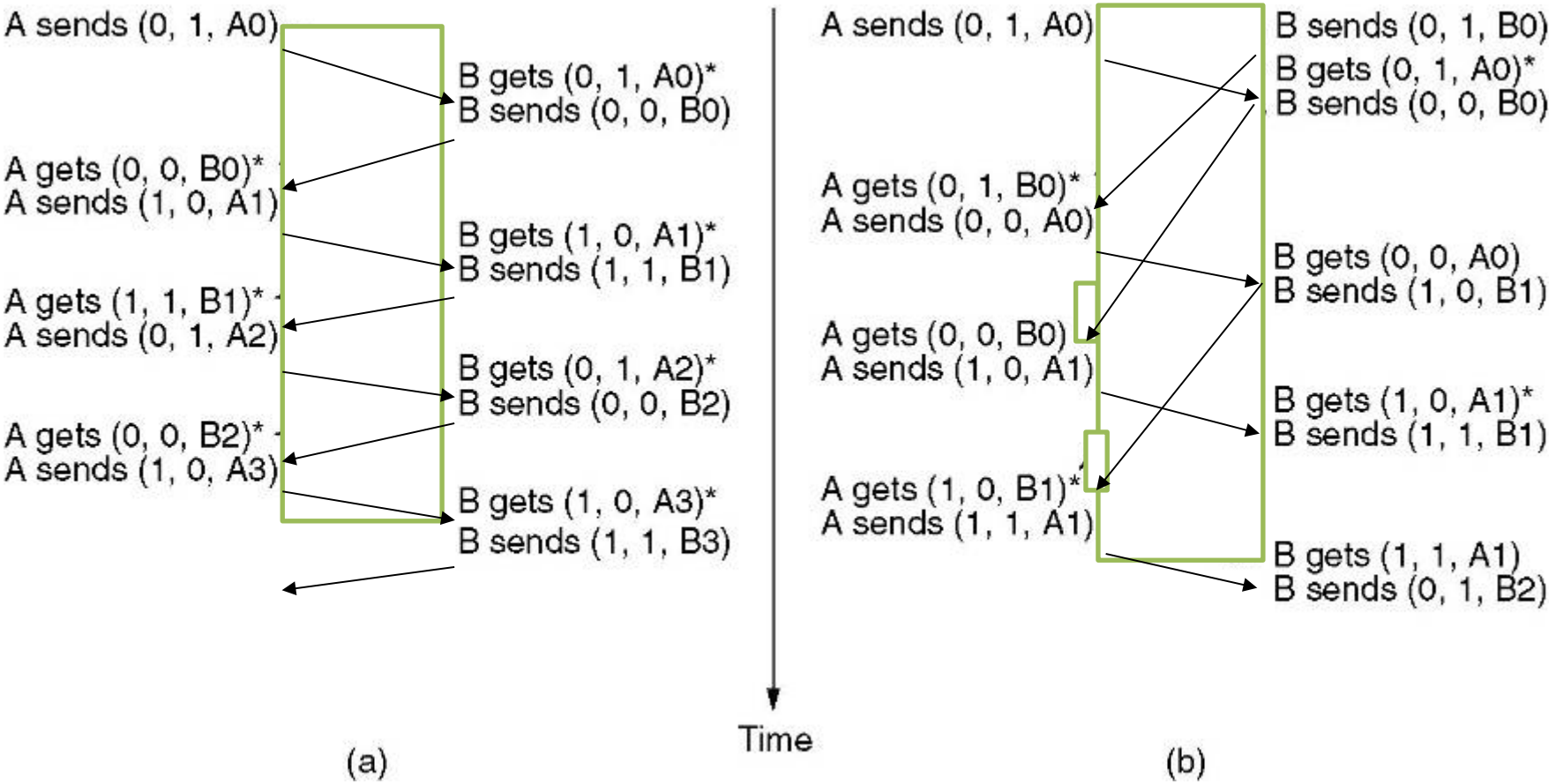
```
do{
  even=wait();
  if(even==SosireCadru){
        r=DeLaFizic();
  //cand este cadrul asteptat, livreaza-l entitatii retea
        if(r.secv==CadruAsteptat){
            LaRetea(r.info);
            inc(CadruAsteptat);
        }
   //cand cadrul transmis este confirmat, pregateste
   // urmatorul cadru
        if(r.conf==CadruUrmator){
            StopCeas(r.conf);
            s.info=DeLaRetea();
            inc(CadruUrmator);
        }
  }
```

//construieste si transmite un cadru

```
s.secv=CadruUrmator;
s.conf=1-CadruAsteptat;
LaFizic(s);
StartCeas(s.secv);
 }forever;    //reia de la asteptarea unui cadru
}
```

Obs. Algoritmul trateaza explicit doar evenimentul SosireCadru.
Este corect? Ce se intampla in celelalte cazuri?

Functionare protocol cu număr de secvență de un bit



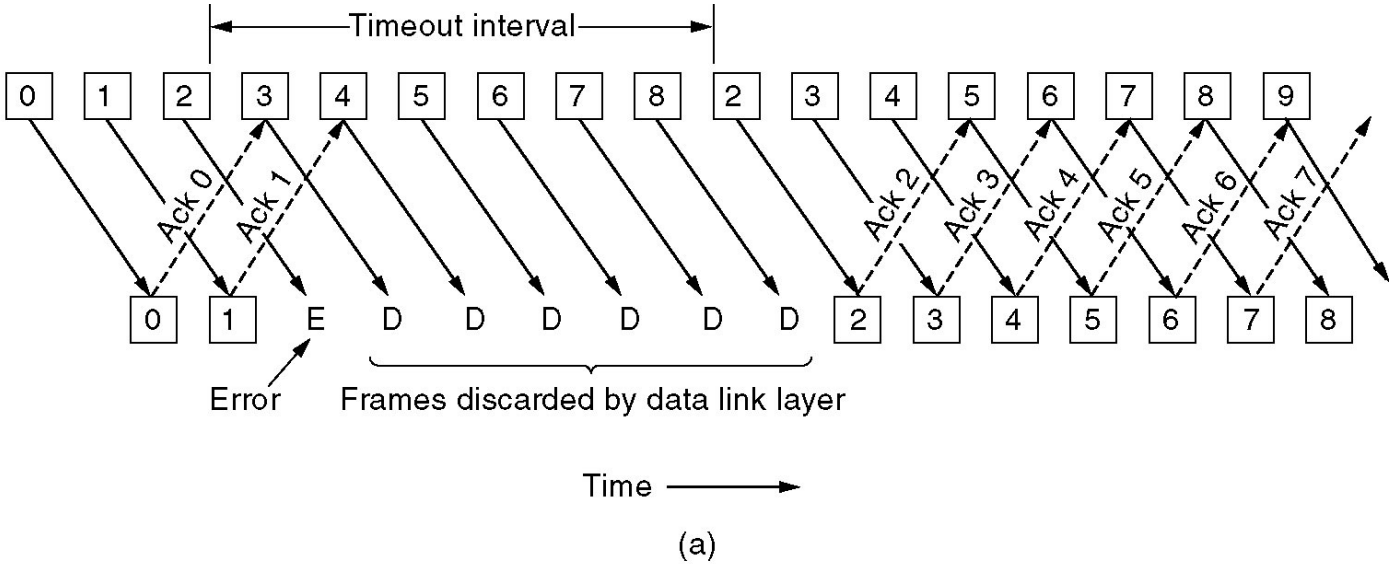
Două scenarii pentru protocolul 4. (a) Cazul normal. (b) Caz anormal.

Notaţia este (secv, conf, identificator packet).

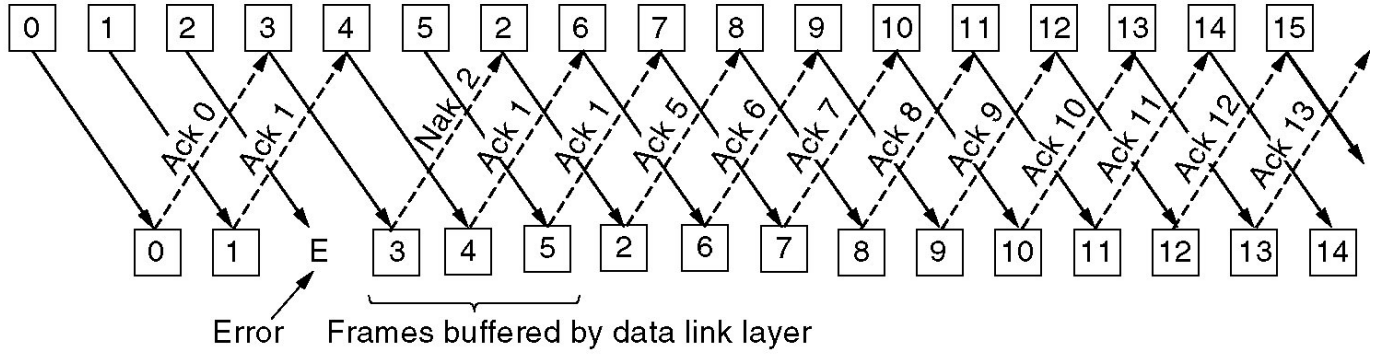
Un asterisc arată că nivelul reţea acceptă pachetul.

Obs. Cum se poate remedia situatia?

Protocoale cu fereastra glisanta

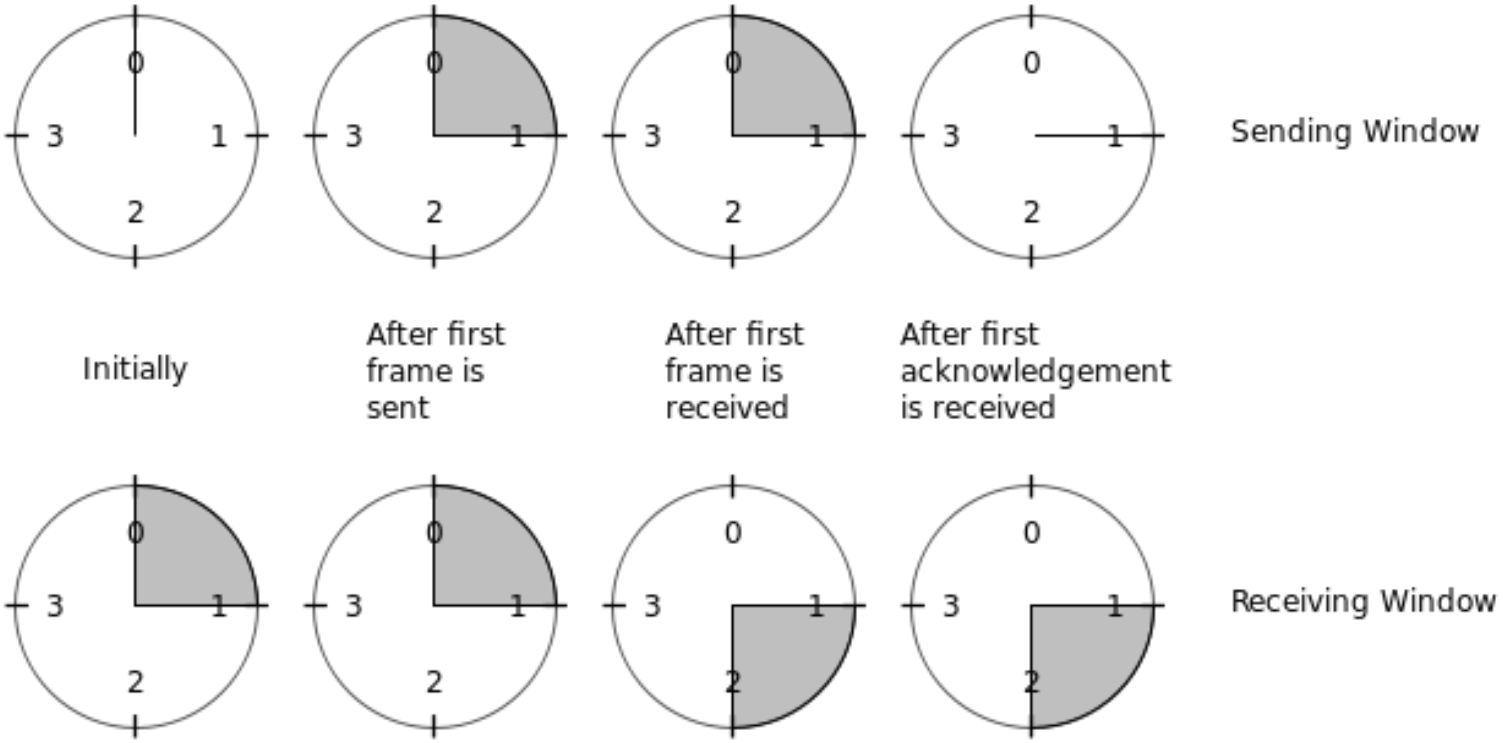


(a)



(b)

Fereastra transmitatorului



A sliding window with a 2-bit sequence, of size 1

Fereastra glisantă

- fereastra este un sub-șir de numere de secvență
  0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …
- pe parcursul transmiterii cadrelor, fereastra glisează
  0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …
- la transmitator, fereastra contine numerele cadrelor transmise si ne-confirmate
- dimensiunea ferestrei transmitatorului este variabila
  - creste cand se trimite un nou cadru; ex. 7
    0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …
  - scade cand se primeste o confirmare; ex. pentru 3 si 4
    0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …

Fereastra receptorului

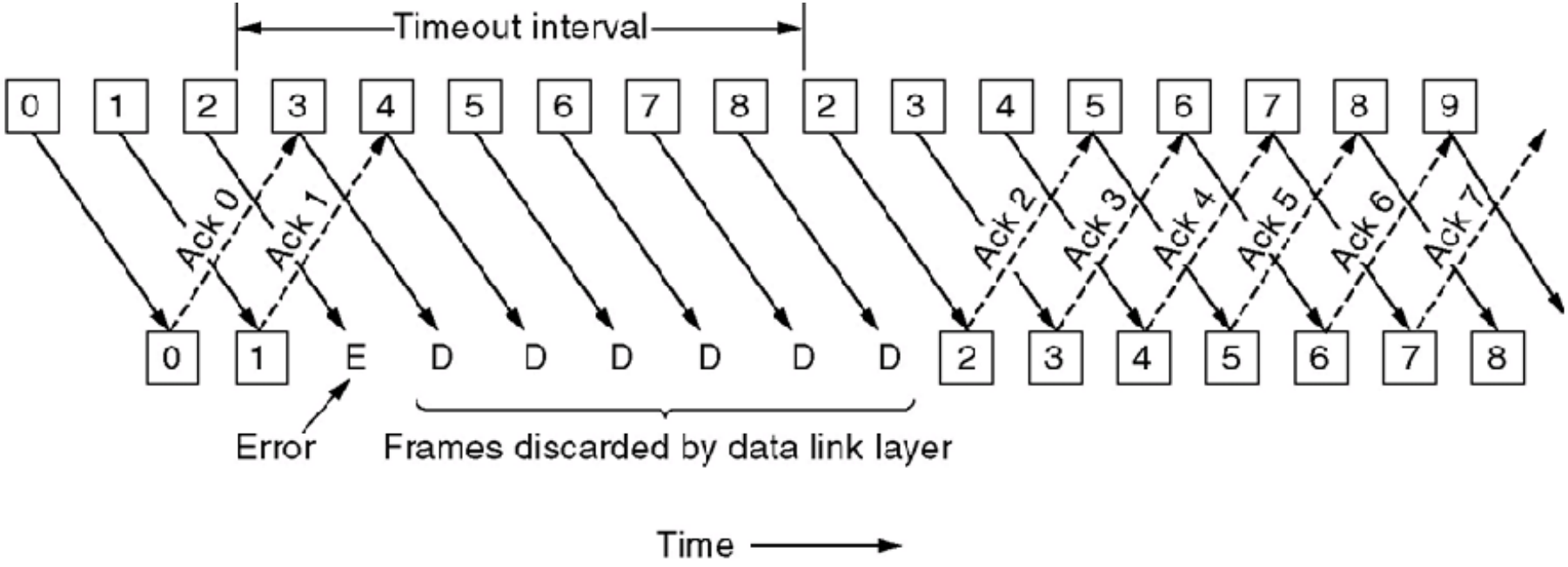
- la receptor, fereastra specifica numerele cadrelor ce pot fi acceptate; in exemplu, se accepta doar cadrele 3, 4, 5, 6, 7
  0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …
- dimensiunea ferestrei receptorului este constanta
- fereastra gliseaza cand unul sau mai multe cadre din stanga ferestrei sunt livrate utilizatorului (livrarea se face in ordinea numerelor de secventa ale cadrelor!)
  - de ex. s-a livrat cadrul 3

  0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …
  - apoi s-au livrat cadrele 4 si 5

  0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …

Un protocol cu retransmitere neselectivă "Go Back N"

Efectul erorii cand fereastra receptorului este 1.



Protocoale cu fereastră supraunitară de transmisie

Protocol cu retransmitere neselectivă

Sunt **`MaxSecv + 1`** numere de secventa diferite
Fereastra maxima a transmitatorului poate fi de **`MaxSecv`** cadre
Demonstratie pe scenariu cu **`MaxSecv = 7`** si fereastra de 8

1. Transmitatorul trimite cadrele 0..7;
2. Toate cadrele sunt receptionate si confirmate;
3. Toate confirmarile sunt pierdute;
4. Transmitatorul retrimite la time-out toate cadrele;
5. Receptorul accepta duplicatele.

```
#define MaxSecv 7
void ActivRetea();
void DezactivRetea();
NrSecv CadruUrmator, //urmatorul cadru de transmis
   ConfAsteptata, //cel mai vechi cadru neconfirmat
      // impreuna desemneaza fereastra transmisie
      CadruAsteptat;    //urmatorul cadru asteptat
cadru r,s;
pachet buffer[MaxSecv+1];
   //index I buffer coincide cu nr de secv. al cadrului din buffer[I]
NrSecv nbuffer,i;
TipEven even;
```

- pachetele sunt pastrate in buffer pana la confirmarea receptiei lor corecte
- astfel, ele pot fi retransmise in caz de eroare
- buffer-ul poate memora un numar finit de pachete
- pentru evitarea depasirii capacitatii buffer-ului, entitatea de legatura de date controleaza preluarea pachetelor de la utilizator (nivelul retea)
  - evenimentul **`ReteaPregatita`** anunta cand utilizatorul are un pachet de transmis
  - functia **`ActivRetea()`** activeaza nivelul retea, care va putea produce evenimente **`ReteaPregatita`**
  - functia **`DezactivRetea()`** activeaza nivelul retea, care nu va putea produce evenimente **`ReteaPregatita`**

```
short intre(NrSecv a, NrSecv b, NrSecv c){
  //intoarce 1 daca a<=b<c circular
  return a<=b && b<c || c<a && a<=b || b<c && c<a;
}
```

ex.1

0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …

a b c

ex.2

0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …

a b c

ex.3

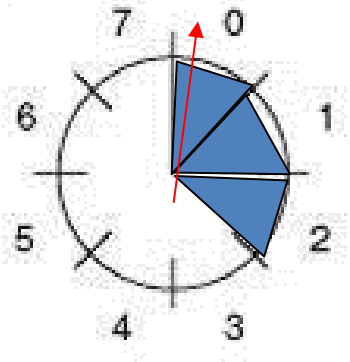
0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1 …

a b c

```
void transmite(NrSecv nrcadru){
   //construieste si transmite un cadru de date
   //pentru pachetul din buffer[nrcadru]

   s.info=buffer[nrcadru];
   s.secv=nrcadru;
   s.conf=(CadruAsteptat+MaxSecv)%(MaxSecv+1);
   //confirmarea trimisa in cadrul de date!
   LaFizic(s);
   StartCeas(nrcadru);
}
```

```
void protocol5(){
   ActivRetea();        // permite even. "ReteaPregatita"
   CadruAsteptat=0;     // fereastra de receptie 1 cadru
   CadruUrmator=0;    // margine sup fereastra transmisie
   ConfAsteptata=0;   // margine inf fereastra transmisie
   nbuffer=0;       // initial nici un pachet in buffer
   do{
     even=wait();         // functionare total dirijata
                 // de evenimente
     switch(even){
        case ReteaPregatita:  // reteaua are de transmis
      //memoreaza pachet in buffer (ptr retransmisie)
      //si trimite-l pe legatura de date
            buffer[CadruUrmator]=DeLaRetea();
               nbuffer++;
               transmite(CadruUrmator);
               inc(CadruUrmator);
               break;
```



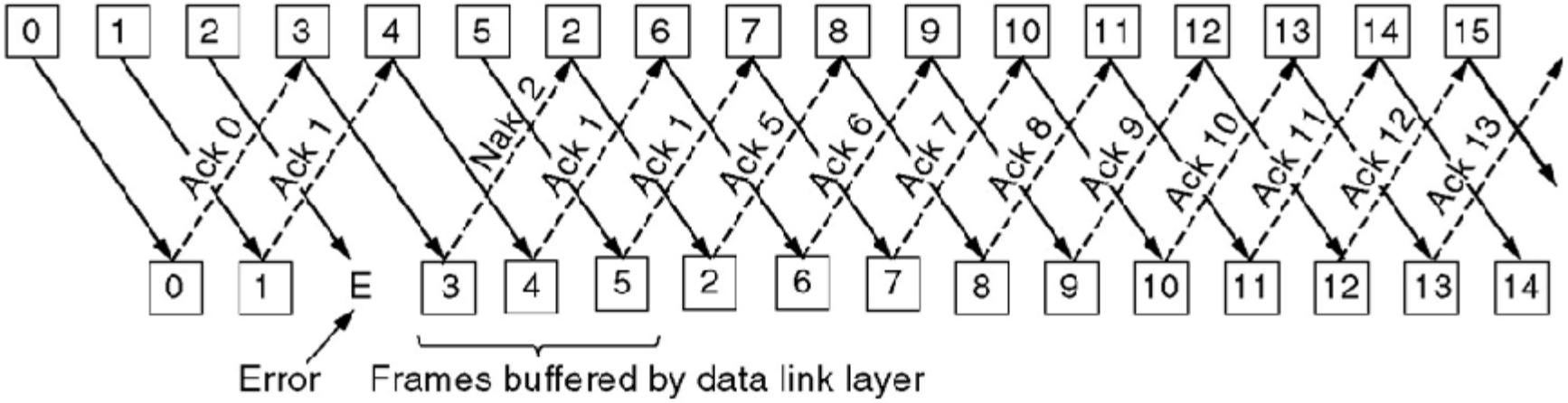

```
case SosireCadru:
   r=DeLaFizic();
   if(r.secv==CadruAsteptat){
      LaRetea(r.info);
      inc(CadruAsteptat);
   }
   while(intre(ConfAsteptata, r.conf, CadruUrmator))
   {
   //confirma cadre intre ConfAsteptata si r.conf
            nbuffer--;
            StopCeas(ConfAsteptata);
            inc(ConfAsteptata);
         }
         break;
    case EroareControl: break;   // ignora cadru eronat
```

```
case TimeOut:  // retransmite toate cadrele din
       // buffer (fereastra transmitatorului)
       //incepand cu pozitia ConfAsteptata
 CadruUrmator=ConfAsteptata;
    for(i=1;i<=nbuffer;i++){
                  transmite(CadruUrmator);
                  inc(CadruUrmator);
          }
}            // sfarsit switch
if(nbuffer<MaxSecv) ActivRetea();
   else DezactivRetea();
 }forever;
}            // sfarsit protocol5
```

Protocol cu retransmitere selectiva “selective repeat”

Fereastra receptorului este supraunitara

- transmite Nak cu numarul 2 pentru cadru eronat
- apoi reconfirma ultimul cadru corect receptionat (Ack 1)
- dupa re-primirea cadrului eronat, Ack 5 confirma toate cadrele pastrate in buffer



Dimensiunea ferestrei de receptie

Fereastra receptorului nu poate fi egală cu cea a transmiţătorului

1. Transmitatorul trimite cadrele 0..6
2. Cadrele sunt receptionate si confirmate. Fereastra receptorului devine 7, 0, 1, 2, 3, 4, 5
3. Toate confirmarile sunt pierdute (se strica sincronizarea intre transmitator si receptor)
4. Transmitatorul retrimite cadrul 0 la time-out
5. Receptorul accepta drept cadru nou aceasta copie (cadrul 0) care se potriveste in fereastra sa actuala (7,0,1,2,3,4,5); cere cadrul 7 (dinaintea lui 0) care lipseste
6. Transmitatorul interpreteaza ca a trimis corect cadrele 0..6 si trimite 7, 0, 1, 2, 3, 4, 5
7. Receptorul accepta cadrele, cu exceptia lui 0, pentru care are deja un cadru receptionat. Ignora acest cadru 0 (a luat în loc duplicatul 0 primit anterior).

```
void protocol6(){
  initializari_contoare;
  do{even=wait();
     switch (even) {
       case ReteaPregatita:
         accepta_salveaza_si_transmite_un_cadru (+-ack);
         //ack se poate include in cadru de date
         //se poate trimite si separat in cadru ack
         break;
```




```
case SosireCadru:
   r=DeLaFizic();
      if (r.fel == data){
         transm_nak_daca_r_difera_de_cadru_asteptat;
         //pentru a semnala rapid eroarea!
         //NU mai multe nak-uri pentru acelasi cadru
         accepta_cadru_daca_in_fereastra_receptie;
         if (sunt date de trimis)
            marcheaza_trebuie_trimis_ack;
         //ack se va trimite cu un cadru de date
         else
         trimite_ack;
         livreaza_in_ordine_pachetele_sosite;
         actualizeaza_fereastra_receptie;
      }
```

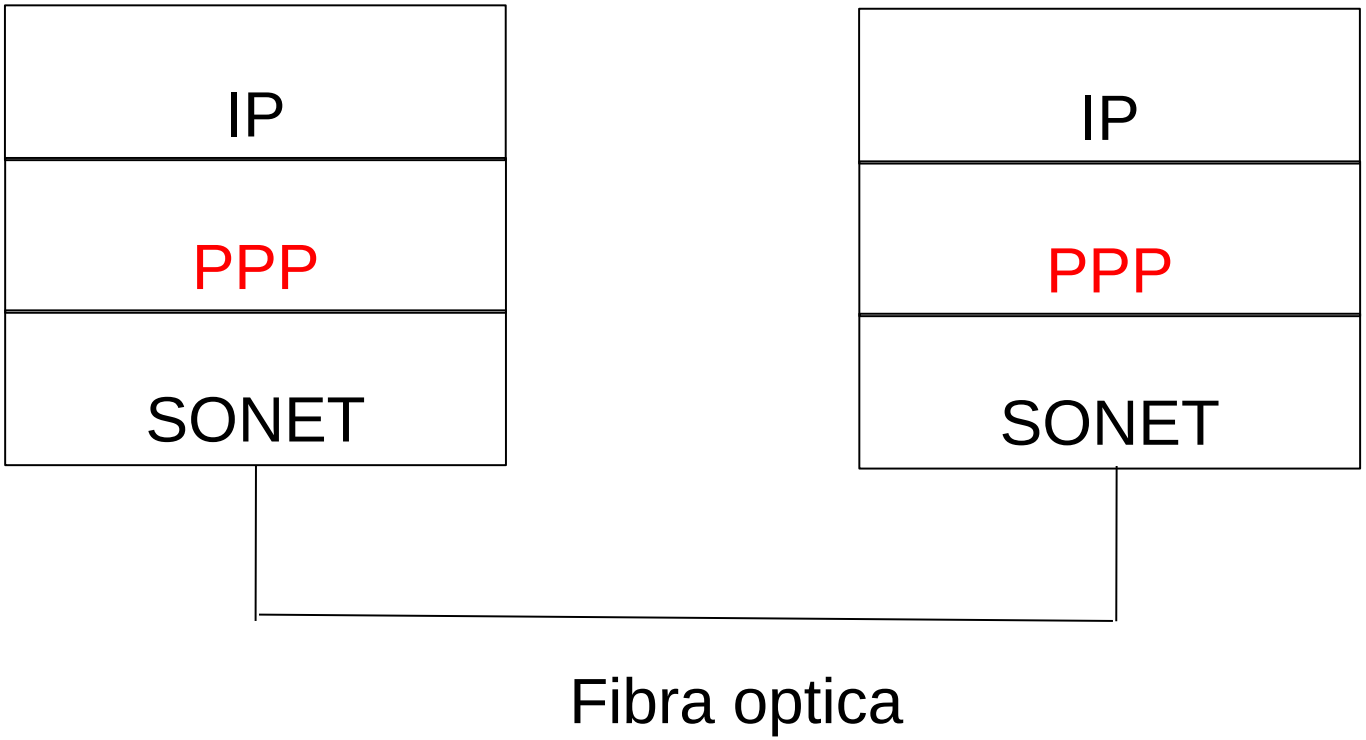
```
//continua case SosireCadru
        if (r.fel == nak) retransmite_cadru_cerut;
        trateaza_confirmare_cadre_eliberind_buffere;
        break;
   case EroareControl:
      transmite_nak;
      break;
   case TimeOut:
      retransmite_cadrul_corespunzator;
      // timeout este asociat nr. secv al unui cadru
   }
   activeaza_sau_dezactiveaza_nivel_retea;
 }forever;
}
```

Exemple Protocoale Data Link

- PPP
- PPPoE

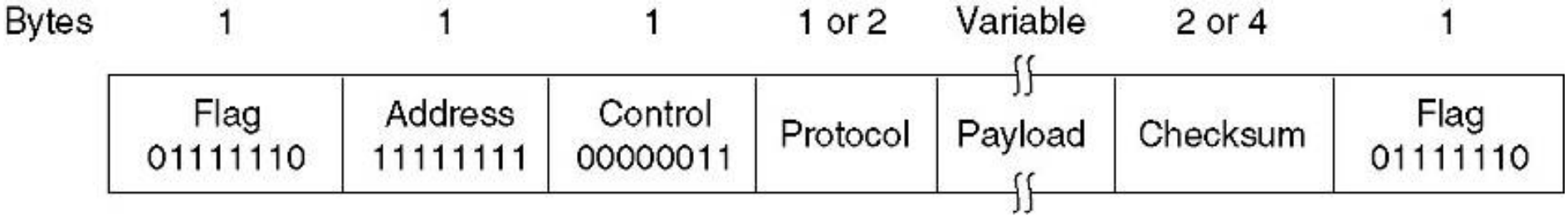
Legatura de date in Internet

Comunicarea pe fibra optica



PPP – Point to Point Protocol

Ofera incadrare
Link Control Protocol, LCP
Network Control Protocol, NCP

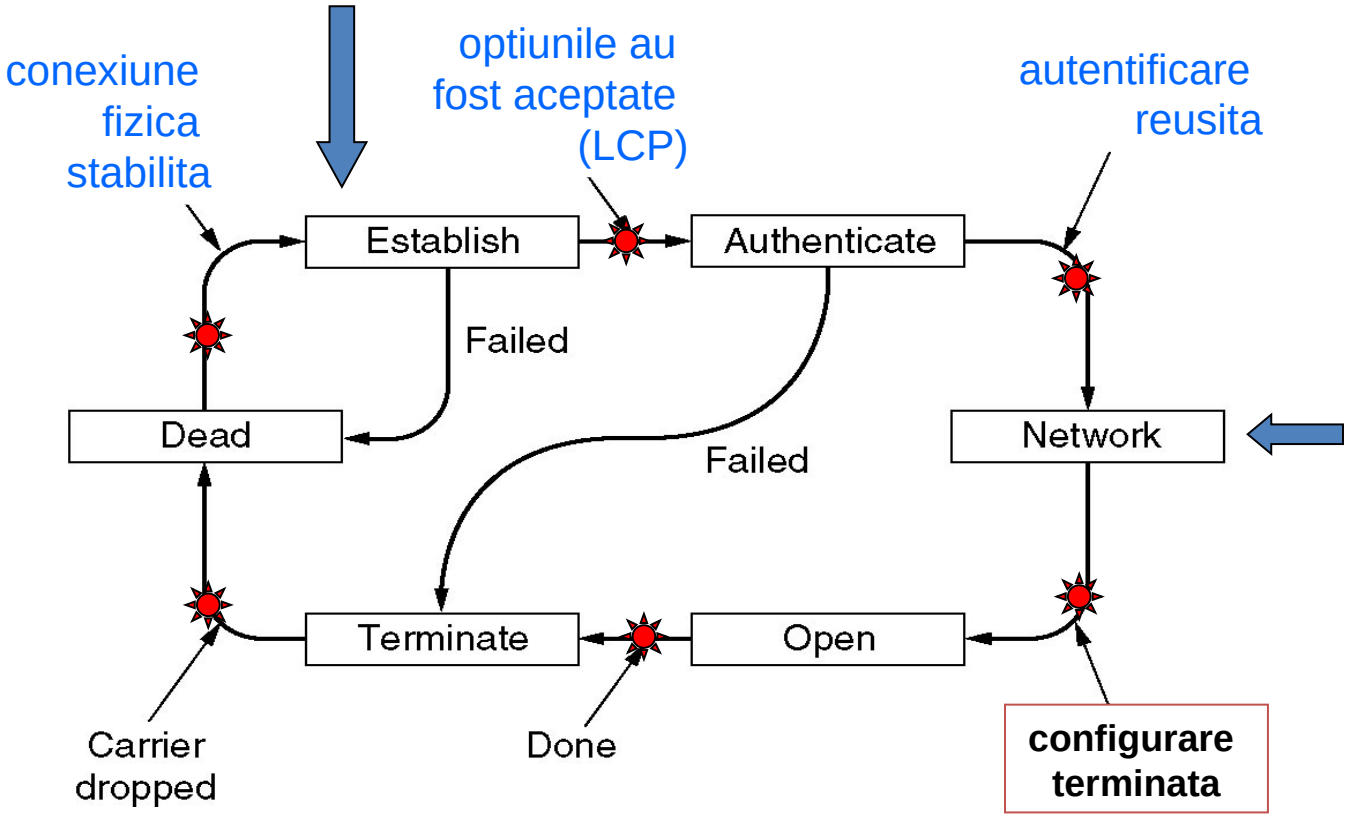


Format de cadru PPP pentru modul nenumerotat

Addresa 11111111 = toate statiile accepta cadrul
Control 00000011 = nenumerotat
Protocol = selecteaza dintre
LCP, NCP
IP, IPX (Internetwork Packet eXchange), OSI CLNP, XNS (Xerox Network Services)

PPP – Point to Point Protocol (2)

Tipuri de cadre LCP
Configure-request **Configure-reject**
Configure-ack **Code-reject**
Configure-nak **Protocol-reject**



protocolul de retea este configurat folosind NCP (ex. ptr IP – alocare adresa IP)

configurare terminata

Inchide conexiune
Terminate-request
Terminate-ack

PPP transporta pachete (de ex. IP)

Tipuri de cadre LCP

Name	Direction	Description
Configure-request	I → R	List of proposed options and values
Configure-ack	I ← R	All options are accepted
Configure-nak	I ← R	Some options are not accepted
Configure-reject	I ← R	Some options are not negotiable
Terminate-request	I → R	Request to shut the line down
Terminate-ack	I ← R	OK, line shut down
Code-reject	I ← R	Unknown request received
Protocol-reject	I ← R	Unknown protocol requested
Echo-request	I → R	Please send this frame back
Echo-reply	I ← R	Here is the frame back
Discard-request	I → R	Just discard this frame (for testing)

I - Initiator **R - Responder**

Point To Point Protocol over Ethernet (PPPoE)

- Publicat in RFC 2516 (1999)

- Extinde capabilitatile protocolului PPP pentru a putea crea o conexiune punct-la-punct peste o retea Ethernet

- Este folosit de catre ISP (Internet Service Provider) pentru a gestiona accesul la retea pentru clienti. Companiile care ofera servicii de Internet pot folosi aceleasi mecanisme de autentificare pentru sesiunile PPPoE si PPP.

- Protocolul este configurat ca o legatura de date punct-la-punct intre doua porturi Ethernet.

Point To Point Protocol over Ethernet (PPPoE)

Pasii protocolului:

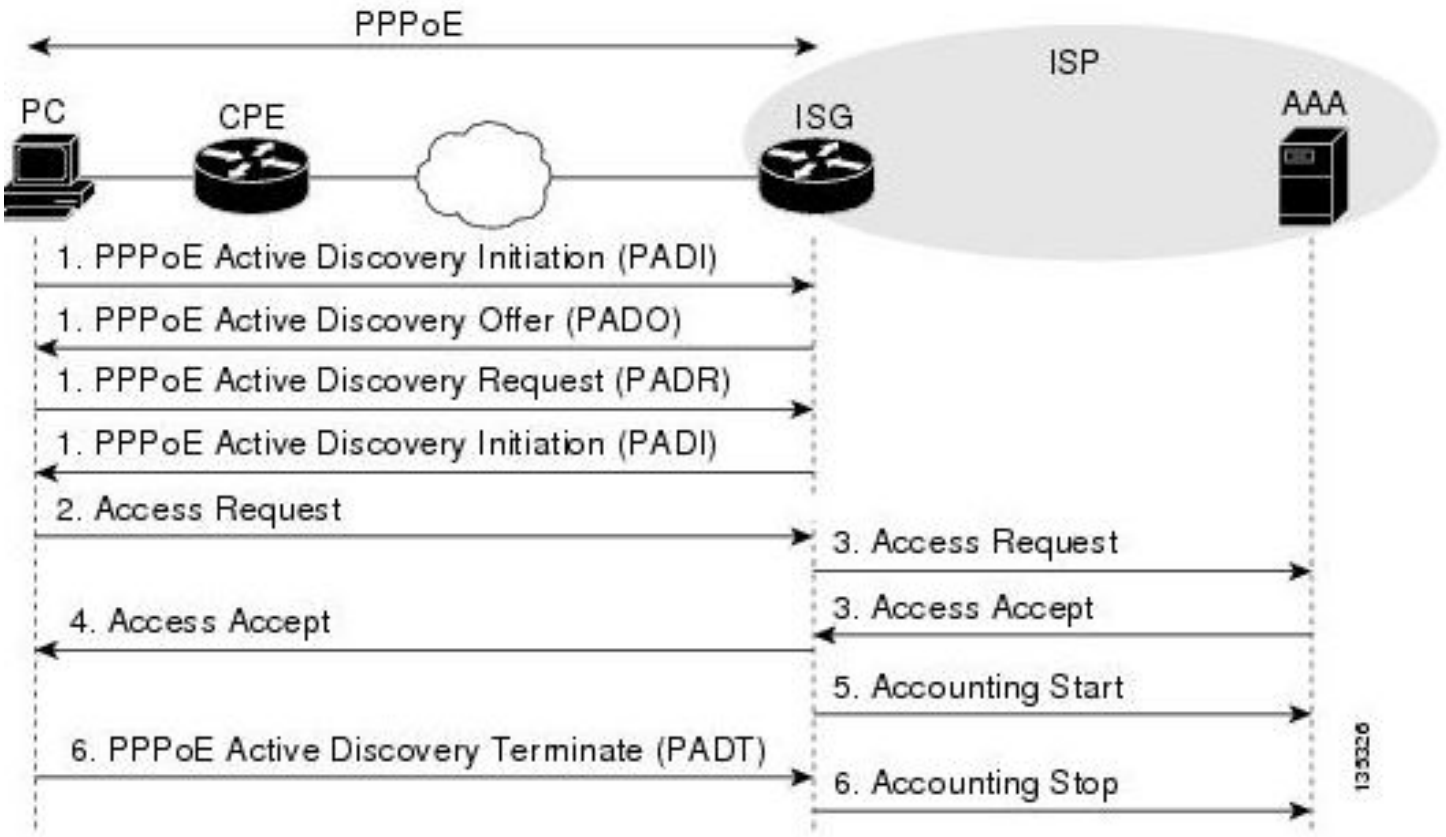
1) Initiation – Clientul trimite un pachet **PPPoE Active Discovery Initiation (PADI)** pentru a initia sesiunea.

2) Offer – Serverul raspunde cu un pachet **PPPoE Active Discovery Offer (PADO).**

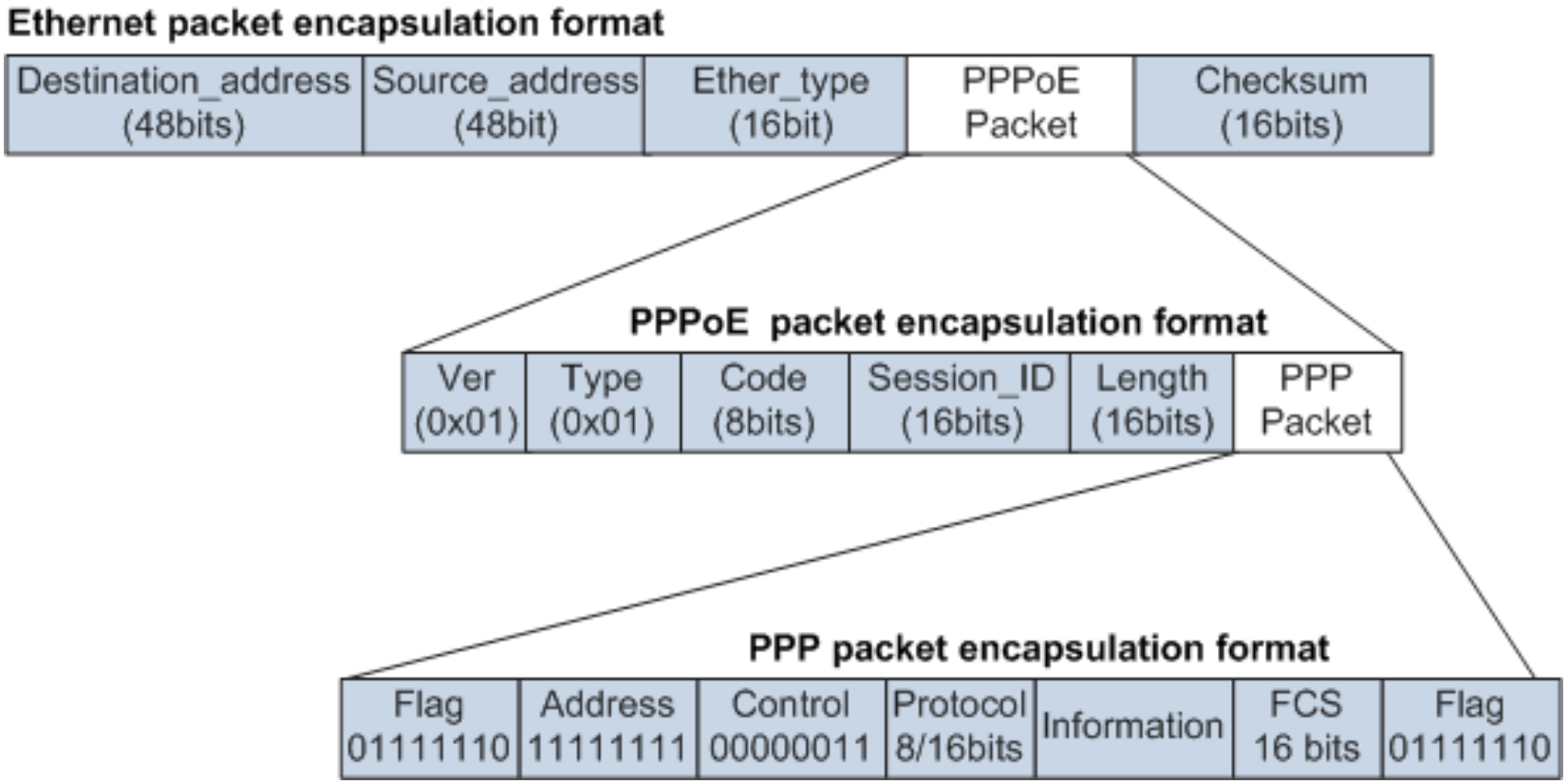
3) Request – Dupa primirea unui pachet PADO, clientul raspunde prin trimiterea unui pachet **PPPoE Active Discovery Request (PADR)** catre server.

4) Confirmation – Dupa primirea unui pachet PADR, serverul raspunde prin generarea unui ID unic pentru sesiunea PPP si trimite un pachet **PPPoE Active Discovery Session (PADS)** de confirmare catre client.

Point To Point Protocol over Ethernet (PPPoE)



Point To Point Protocol over Ethernet (PPPoE)



Point To Point Protocol over Ethernet (PPPoE)

Field	Description
Destination_address	Indicates either a unicast Ethernet destination address or the Ethernet broadcast address (0xffffffffffff). For Discovery packets, the value is either a unicast or broadcast address. A PPPoE client uses a broadcast address to search for a PPPoE server and uses a unicast address after choosing a PPPoE server. For PPP session traffic, this field must contain the peer's unicast address as determined from the Discovery stage.
Source_address	Indicates the Ethernet MAC address of a source device.
Ether_type	Set to either 0x8863 (Discovery stage) or 0x8864 (Session stage).
Ver	Indicates a PPPoE version number. The **Ver** field is four bits and must be set to 0x1.
Type	Indicates a PPPoE type. The **Type** field is four bits and must be set to 0x1.
Code	Indicates a PPPoE packet type. The **Code** field is eight bits and has different values: • 0x00: session data • 0x09: PADI packet • 0x07: PADO or PADT packet • 0x19: PADR packet • 0x65: PADS packet
Session_ID	The **SESSION_ID** field is sixteen bits. The value is fixed for a given PPP session and defines a PPP session along with Ethernet source and destination addresses. A value of 0xffff is reserved for future use and must not be used.
Length	Indicates the length of the PPPoE payload. The **Length** field is 16 bits, excluding the length of the Ethernet or PPPoE header.
PPP Packet	For PPP packet format, see PPP Basic Concepts-PPP Packet Format.

Studiu individual

A. S. Tanenbaum Reţele de calculatoare, ed 4-a, BYBLOS 2003
3.1 ASPECTE ALE PROIECTĂRII NIVELULUI LEGĂTURĂ DE DATE
3.2.2 Coduri detectoare de erori
3.3 PROTOCOALE ELEMENTARE PENTRU LEGĂTURA DE DATE
3.4 PROTOCOALE CU FEREASTRĂ GLISANTĂ
3.6 EXEMPLE DE PROTOCOALE ALE LEGĂTURII DE DATE
A. S. Tanenbaum Computer networks, 5-th ed. PEARSON 2011
3.1 DATA LINK LAYER DESIGN ISSUES
3.2.2 Error-Detecting Codes
3.3 ELEMENTARY DATA LINK PROTOCOLS
3.4 SLIDING WINDOW PROTOCOLS
3.5.1 Packet over SONET